FINANCIAL RISK PLATFORM

WS2 | Backend Development Plan

Data Ingestion, Mapping & Data Quality

Workstream objective: Accept uncertain source shapes safely through configurable contracts, mappings and validation rather than hardcoding one client file format.

Product intent: A broad, lightweight, extensible platform foundation. Initial data volumes and module depth are limited deliberately; architecture, control points and expansion interfaces are not treated as throwaway demo code.

> **Synchronization rule: This document is paired with the opposite-side document for the same workstream. Feature IDs, phase IDs, endpoint contracts, statuses and acceptance criteria must remain aligned.**

Shared architecture: Layer 1 Universal Platform Foundation → Layer 2 Configurable Financial Domain → Layer 3 Financial Calculation Modules → Layer 4 Intelligence and Experience. Data governance and security cross-cut all layers.

1. Scope and Feature Coverage

Feature ID	Feature	Scope / implementation intent
DI-01	Source registration	Register file/feed/source type, owner, cadence and expected contract when known.
DI-02	Flexible ingestion	Accept bounded CSV/XLSX/JSON samples initially; adapter interfaces allow future warehouse/API/batch feeds.
DI-03	Source-to-canonical mapping	Versioned mappings from unknown source attributes to canonical fields.
DI-04	Validation and quarantine	Field-level validation, rejected rows, warnings and quarantine/retry outcomes.
DI-05	Reconciliation	Record counts, totals and source-to-target comparison where applicable.
DI-06	Data profiling and drift	Infer candidate types and flag shape/distribution drift without silently changing the model.

2. Phase-Wise Delivery

P0 | Sample contracts and uncertainty policy

Create synthetic sample shapes and mark assumptions as provisional.

P1 | Source and import jobs

Register source profiles, create import jobs, track status and file metadata.

P2 | Mapping workbench

Map source fields to canonical fields; version and validate mapping configurations.

P3 | Quality, quarantine and reconciliation

Return actionable issues, rejected-row references, counts and reconciliation outcomes.

P4 | Profiling and drift detection

Suggest field types and detect changed columns/value patterns; suggestions require review.

P5 | Adapter readiness

Document and test adapter contract for future warehouse/API/event source integrations.

3. Implementation Responsibilities

Backend responsibilities

• Create import-job service with safe file limits, checksum/metadata, status and stable job ID.

• Implement adapter interface independent of the first file formats; use CSV/XLSX/JSON as initial demo adapters.

• Persist versioned source profiles and source-to-canonical mapping definitions.

• Validate types, required fields, dates, currencies, precision, duplicates, references and configurable rules.

• Quarantine or reject invalid data according to policy; never silently drop rows.

• Provide profiling/drift suggestions and reconciliation results without automatically altering canonical schema.

4. Paired API / Feature Contract Matrix

Endpoint names are proposed contracts for alignment, not a claim that the APIs already exist. Confirm payload schemas in the shared OpenAPI contract before implementation.

Shared feature pair	Capability	Proposed API contract
FE-DI-01 / BE-DI-01	Source/import wizard	POST /api/v1/sources; POST /api/v1/imports; GET /api/v1/imports/{import_id}
FE-DI-02 / BE-DI-02	Field profiling	GET /api/v1/imports/{import_id}/profile
FE-DI-03 / BE-DI-03	Mapping workbench	GET /api/v1/mappings; POST /api/v1/imports/{import_id}/mappings/validate; POST /api/v1/mappings
FE-DI-04 / BE-DI-04	Quality/quarantine	POST /api/v1/imports/{import_id}/validate; GET /api/v1/imports/{import_id}/issues; GET /api/v1/quarantine/{batch_id}
FE-DI-05 / BE-DI-05	Reconciliation/drift	GET /api/v1/imports/{import_id}/reconciliation; GET /api/v1/imports/{import_id}/drift

5. Cross-Cutting Rules

• The canonical model and source mapping are versioned separately. Unknown source attributes are preserved or quarantined by explicit policy, not silently discarded.

• Every calculation run must bind dataset snapshot, schema/mapping version, rule-set version and calculation-engine version.

• Server-side authorization is authoritative. The frontend may hide unavailable actions but does not enforce security.

• No AI-generated code is executed. AI output is validated against schemas and policy, and consequential actions require authorized approval.

• All errors use a structured response with code, message, details and correlation_id. Sensitive payloads and secrets must not be logged.

• Prototype financial modules use synthetic/approved data, documented assumptions and test fixtures; do not claim regulatory certification or Moody's parity.

• The platform is designed for breadth, but production-grade depth is not implied for every financial domain. ALM/IRRBB, collateral, capital, live market data and regulatory feeds require their own specifications and validation.

6. Definition of Done

• Every feature ID in this document has a matching feature ID in its paired frontend/backend document and in the shared feature register.

• Each endpoint has request/response schemas, status enums, authorization expectations and error examples in OpenAPI.

• Frontend components use actual API responses or clearly labelled mocks matching the contract.

• Backend tests cover validation, permissions, version binding and domain logic appropriate to this workstream.

• The cross-workstream integration test passes for the features included in the release.

Change-control rule: Any change to endpoint, field name, enum, versioning behaviour or acceptance criteria must be reflected in both paired documents and the shared contract in the same change.